

OpenTelemetry (OTel)

Complete Setup Guide

Operator · Collector · Instrumentation · Application Onboarding

1. Architecture Overview

The OpenTelemetry stack consists of four main components that work together to collect, process, and forward traces from your applications to the central Tempo backend.

Component	Namespace	Role
OTel Operator	observability-poc	Manages OTel resources — creates Collector pods and injects agents into application pods
OTel Collector	observability-poc	Receives traces from apps, processes them, and forwards to central Tempo
Instrumentation CR	observability-poc	Defines agent configuration — endpoint, sampler, language-specific settings
Your Application	tevico	Gets auto-instrumented via annotation — no code changes needed

Data Flow

Your Application -> OTel Collector -> Central Tempo -> Grafana

NOTE: All four components must be set up in order. Do not skip any step.

2. Prerequisites

Before starting, ensure the following are available:

- kubectl configured and connected to the Preprod cluster
- Helm v3.x installed
- Access to observability-poc namespace
- Access to <running application> namespace (for application onboarding)

Verify your cluster access:

```
kubectl cluster-info  
kubectl get namespaces
```

Create the namespace if it does not exist:

```
kubectl create namespace observability-poc
```

3. Step 1 — Install OTel Operator

The OTel Operator is a Kubernetes Operator that manages OpenTelemetry resources. It must be installed FIRST before the Collector and Instrumentation CR.

3.1 Create Chart Directory

```
mkdir -p otel-operator && cd otel-operator
```

3.2 Create Chart.yaml

```
apiVersion: v2  
name: otel-operator
```

```
description: OpenTelemetry Operator
type: application
version: 1.0.0
appVersion: 1.0.0
dependencies:
  - name: opentelemetry-operator
    version: 0.76.1
    repository: https://open-telemetry.github.io/opentelemetry-helm-charts
    condition: operator.enabled
    alias: operator
```

3.3 Create values.yaml

```
operator:
  enabled: true
  fullnameOverride: otel-operator
  additionalLabels:
    app: otel-operator
  replicaCount: 2
  manager:
    image:
      repository:
ghcr.io/open-telemetry/opentelemetry-operator/opentelemetry-operator
      tag: 0.105.0
    collectorImage:
      repository:
ghcr.io/open-telemetry/opentelemetry-collector-releases/opentelemetry-collector-contrib
      tag: 0.126.0
    autoInstrumentationImage:
      java:
        repository:
ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-java
        tag: 1.33.6
      nodejs:
        repository:
ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-nodejs
        tag: 0.58.0
      python:
        repository:
ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-python
        tag: 0.59b0
      dotnet:
        repository:
ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-dotnet
        tag: 1.2.0
    serviceMonitor:
      enabled: true
  admissionWebhooks:
```

```
certManager:
  enabled: false
autoGenerateCert:
  enabled: true
  recreate: false
  certPeriodDays: 3650
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app.kubernetes.io/name: operator
            app: otel-operator
        topologyKey: kubernetes.io/hostname
```

3.4 Install

```
helm dependency update .
helm upgrade --install otel-operator . -n observability-poc -f values.yaml
```

3.5 Verify

```
kubectl get pods -n observability-poc | grep otel-operator
```

Expected output (2 pods, both 2/2 Running):

```
otel-operator-xxxxxxxx-xxxxx 2/2 Running 0 2m
otel-operator-xxxxxxxx-xxxxx 2/2 Running 0 2m
```

4. Step 2 — Install OTel Collector

The OTel Collector receives traces from all application pods, adds metadata (tenant ID, k8s labels), and forwards them to the central Tempo backend. Install after the Operator.

4.1 Create Chart Directory

```
mkdir -p otel-collector && cd otel-collector
```

4.2 Create Chart.yaml

```
apiVersion: v2
name: otel-collector
description: OpenTelemetry Collector
type: application
version: 1.0.0
appVersion: 0.124.1
dependencies:
  - name: opentelemetry-kube-stack
    version: 0.12.4
    repository: https://open-telemetry.github.io/opentelemetry-helm-charts
    condition: collector.enabled
    alias: collector
    tags:
      - collector
```

4.3 Create values.yaml

NOTE: Replace <YOUR_TENANT_ID> and <BASE64_CREDENTIALS> with values provided by team.

```
collector:
  enabled: true
  crds:
    enabled: false
  opentelemetry-operator:
    enabled: false
  collectors:
    daemon:
      enabled: false
  otel:
    mode: deployment
    suffix: otel
    enabled: true
    replicas: 1
    image:
      repository:
ghcr.io/open-telemetry/opentelemetry-collector-releases/opentelemetry-collector-contrib
      tag: 0.140.1
    resources:
      limits:
```

```

    cpu: 200m
    memory: 500Mi
  requests:
    cpu: 100m
    memory: 250Mi
  config:
    extensions:
      health_check:
        endpoint: 0.0.0.0:13133
    receivers:
      otlp:
        protocols:
          grpc:
            endpoint: 0.0.0.0:4317
          http:
            endpoint: 0.0.0.0:4318
    processors:
      batch:
        timeout: 5s
        send_batch_size: 1
      resource:
        attributes:
          - key: tenant.id
            value: <YOUR_TENANT_ID>      # e.g. pre-prod-tevico-cluster
            action: upsert
    exporters:
      otlphttp/tempo:
        endpoint: https://grafana-central.tevi.co/olly/tempo/
        tls:
          insecure: false
          insecure_skip_verify: true
        timeout: 30s
        headers:
          X-Scope-OrgID: <YOUR_TENANT_ID>      # e.g.
pre-prod-tevico-cluster
          Authorization: Basic <BASE64_CREDENTIALS>
      debug:
        verbosity: basic
    service:
      extensions: [health_check]
      telemetry:
        metrics:
          level: none
      pipelines:
        traces:
          receivers: [otlp]
          processors: [batch, resource]
          exporters: [otlphttp/tempo, debug]
  instrumentation:
    enabled: false

```

4.4 Install

```
helm dependency update .
helm upgrade --install otel-collector . -n observability-poc -f values.yaml
```

4.5 Verify

```
kubectl get pods -n observability-poc | grep otel-collector
```

Expected output:

```
otel-collector-collector-xxxxxxxxx-xxxxx 1/1 Running 0 1m
```

5. Step 3 — Create Instrumentation CR

The Instrumentation CR defines how agents are configured when injected into application pods. It must be created in the observability-poc namespace.

5.1 Create instrumentation.yaml

```
apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: otel-instrumentation
  namespace: observability-poc
spec:
  exporter:
    endpoint:
      http://otel-collector-collector.observability-poc.svc.cluster.local:4317
  propagators:
    - tracecontext
    - baggage
    - b3
  sampler:
    argument: '1'
    type: parentbased_traceidratio
  resource:
    addK8sUIDAttributes: true
  java:
```

```

    image:
ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-java:1.33.6
  env:
    - name: OTEL_METRICS_EXPORTER
      value: none
    - name: OTEL_LOGS_EXPORTER
      value: none
  resources:
    limits:
      cpu: 500m
      memory: 256Mi
    requests:
      cpu: 50m
      memory: 64Mi
  python:
    image:
ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-python:0.60b1
  env:
    - name: OTEL_METRICS_EXPORTER
      value: none
    - name: OTEL_LOGS_EXPORTER
      value: none
  nodejs:
    image:
ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-nodejs:0.58.0
  env:
    - name: OTEL_METRICS_EXPORTER
      value: none
    - name: OTEL_LOGS_EXPORTER
      value: none

```

5.2 Apply

```
kubectl apply -f instrumentation.yaml
```

5.3 Verify

```
kubectl get instrumentation -n observability-poc
```

Expected output:

NAME	AGE	ENDPOINT
otel-instrumentation	30s	http://otel-collector-collector.observability-poc...:4317

6. Step 4 — Onboard Your Application

Once all three components are running, you can onboard any application by adding labels and an annotation to its Deployment. No code changes are required.

6.1 Add Required Labels

Labels must be added to your pod template so that traces and metrics can be filtered by environment, service, and cluster in Grafana dashboards.

Edit your Deployment YAML or run:

```
kubectl edit deployment <your-deployment-name> -n <namespace>
```

Add labels under `spec.template.metadata.labels`:

```
spec:
  template:
    metadata:
      labels:
        app: <your-app-name>
        comprinno_environment: preprod
        comprinno_service: <your-service-name>
        comprinno_application_group: <your-group-name>
        comprinno_cluster: pre-prod-tevico-cluster
```

Label	Example Value	Purpose
app	dashboard	Application name identifier
comprinno_environment	preprod	Environment name — used in Grafana dropdown filters
comprinno_service	dashboard	Service name — used in Grafana dropdown filters
comprinno_application_group	comprinno	Application group — used in Grafana dropdown filters
comprinno_cluster	pre-prod-tevico-cluster	Cluster name — used in Grafana dropdown filters

NOTE: All five labels are required. Without them, traces will still flow but Grafana dashboard filters will not populate correctly.

6.2 Add Auto-Instrumentation Annotation

Add this annotation under `spec.template.metadata.annotations` to enable automatic agent injection:

```
spec:
  template:
    metadata:
      annotations:
        instrumentation.opentelemetry.io/inject-java:
observability-poc/otel-instrumentation
```

Use the correct annotation key for your language:

Language	Annotation Key	Annotation Value
Java	instrumentation.opentelemetry.io/inject-java	observability-poc/otel-instrumentation
Python	instrumentation.opentelemetry.io/inject-python	observability-poc/otel-instrumentation
Node.js	instrumentation.opentelemetry.io/inject-nodejs	observability-poc/otel-instrumentation
.NET	instrumentation.opentelemetry.io/inject-dotnet	observability-poc/otel-instrumentation

6.3 Complete Deployment Example

Below is a complete, ready-to-use example for a Java application:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dashboard
  namespace: tevico
spec:
  replicas: 2
  selector:
    matchLabels:
      app: dashboard
  template:
    metadata:
      labels:
```

```
    app: dashboard
    comprinno_environment: preprod
    comprinno_service: dashboard
    comprinno_application_group: comprinno
    comprinno_cluster: pre-prod-tevico-cluster
    annotations:
      instrumentation.opentelemetry.io/inject-java:
observability-poc/otel-instrumentation
  spec:
    containers:
      - name: dashboard
        image: your-registry/dashboard:latest
        ports:
          - containerPort: 8080
```

6.4 Restart the Deployment

```
kubectl rollout restart deployment/<your-deployment-name> -n namespace
kubectl rollout status deployment/<your-deployment-name> -n namespace
```

NOTE: This is a rolling restart. Old pods will stay running until new pods are ready. Zero downtime.

7. Step 5 — Verify Everything is Working

7.1 Verify Agent Was Injected

```
kubectl describe pod <pod-name> -n namespace | grep -A 8 'Init Containers'
```

Expected output:

```
Init Containers:
  opentelemetry-auto-instrumentation-java:
    Image:   ghcr.io/.../autoinstrumentation-java:1.33.6
    State:   Terminated (Reason: Completed)
```

Verify OTEL environment variables:

```
kubectl describe pod <pod-name> -n namespace | grep OTEL
```

Expected output:

```
OTEL_SERVICE_NAME:      dashboard
OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-collector-collector...:4317
OTEL_TRACES_EXPORTER:   otlp
OTEL_METRICS_EXPORTER:  none
OTEL_LOGS_EXPORTER:     none
```

7.2 Verify Traces in Collector

```
kubectl logs -n observability-poc \
$(kubectl get pod -n observability-poc -l app.kubernetes.io/component=otel \
-o jsonpath='{.items[0].metadata.name}') --tail=20
```

You should see trace entries like:

```
info Traces {otelcol.component.id: debug, resource spans: 1, spans: 3}
```

7.3 Verify Traces in Grafana

1. Open Grafana: <https://grafana-central.tevi.co>
2. Select organization: pre-prod-tevico-cluster (top left dropdown)
3. Go to Explore -> Select datasource: tempo-preprod
4. Query type: Search -> Service Name: select your service
5. Click Run query — traces should appear

INFO: If traces do not appear immediately, wait 1-2 minutes and try again.

8. Troubleshooting

8.1 Init Container Not Injected

- Verify annotation spelling is correct — no typos
- Verify Operator is running: `kubectl get pods -n observability-poc | grep operator`
- Check Operator logs:

```
kubectl logs -n observability-poc <otel-operator-pod> -c manager --tail=20 | grep -i inject
```

- Verify Instrumentation CR exists: `kubectl get instrumentation -n observability-poc`
- Ensure pod was restarted AFTER the annotation was added

8.2 Traces Not in Grafana

- Check Collector logs for export errors:

```
kubectl logs -n observability-poc <collector-pod> --tail=30 | grep -i error
```

- Make sure your application is receiving actual HTTP traffic
- Set Grafana time range to Last 1 hour or wider
- Confirm you selected the correct Grafana organization

8.3 Application Fails to Start After Annotation

- Check pod logs: `kubectl logs <pod-name> -n tevico`
- Look for port conflicts — OTel agent uses port 8888 internally
- Check if memory limits need to be increased (Java agent needs ~256Mi extra)

9. Quick Reference

Installation Order

6. Install OTel Operator (Step 3)
7. Install OTel Collector (Step 4)
8. Create Instrumentation CR (Step 5)
9. Add labels to your Deployment (Step 6.1)
10. Add annotation to your Deployment (Step 6.2)
11. Restart your Deployment (Step 6.4)
12. Verify in Grafana (Step 7.3)

Required Labels Summary

Label Key	Value	Required
app	your-app-name	Yes
comprinno_environment	preprod	Yes
comprinno_service	your-service-name	Yes
comprinno_application_group	your-group-name	Yes
comprinno_cluster	pre-prod-tevico-cluster	Yes

Annotation by Language

Language	Annotation Key
Java	instrumentation.opentelemetry.io/inject-java
Python	instrumentation.opentelemetry.io/inject-python
Node.js	instrumentation.opentelemetry.io/inject-nodejs
.NET	instrumentation.opentelemetry.io/inject-dotnet

Annotation value is always: observability-poc/otel-instrumentation
